Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Abstraction, Packages, Encapsulation
Constructors, Overloading

Παναγιώτης Ηλία

Παράδειγμα

```
import java.util.Scanner;

class Car
{
    private int position = 0;

    public boolean moveManySteps(int steps)
    {
        if ((position + steps < -10) || (position + steps > 10)){
            return false;
        }
        position += steps;
        return true;
    }
    public void printPosition(){
        System.out.println("Car at position "+position);
    }
}

class MovingCar4c
{
    public static void main(String args[]){
        Scanner input = new Scanner(System.in);
        Car myCar = new Car();
        int steps = input.nextInt();
        myCar.moveManySteps(steps);
        myCar.printPosition();
    }
}
```

Παράδειγμα

- Όταν καλούμε την συνάρτηση move() το όχημα μας θα κινείται ένα τυχαίο αριθμό από βήματα στο διάστημα (-3,3)

Υλοποίηση

- Θα φτιάξουμε μια βοηθητική συνάρτηση που θα μας επιστρέφει τον τυχαίο αριθμό από βήματα.

private: Δεν χρειάζεται να φαίνεται έξω από την κλάση

```
private int computeRandomSteps()
{
     int radomSteps;
     // do the computation

     return randomSteps;
}

public void move(){
     int steps = computeRandomSteps();
     moveManySteps(steps);
}
```

Κλήση της συνάρτησης και χρήση της επιστρεφόμενης τιμής

```
import java.util.Random;

class Car
{
  private int MAX_VALUE = 3;
  private int position = 0;
  private Random randomGenerator = new Random();

  private int computeRandomSteps()
  {
      int randomSteps = randomGenerator.nextInt(2*MAX_VALUE + 1) - MAX_VALUE;
      return randomSteps;
  }

  public void move(){
      int steps = computeRandomSteps();
      moveManySteps(steps);
  }

  public void moveManySteps(int steps) { ... }

  public void printPosition(){
      System.out.println("Car at position "+position);
  }
}

class MovingCar5
{
  public static void main(String args[]){
    Car myCar = new Car();
    myCar.move();
  }
}
```

Η κλάση Random: Δημιουργεί μια γεννήτρια τυχαίων αριθμών που παράγει τυχαίους αριθμούς

Μέθοδος nextInt(int x) της Random: Επιστρέφει ένα τυχαίο ακέραιο αριθμό στο διάστημα [0, x)

Public/Private

- Ότι είναι ορισμένο ως public σε μία κλάση είναι προσβάσιμο από οποιονδήποτε
  - Μπορούμε να καλέσουμε τις μεθόδους ορίζοντας ένα αντικείμενο της κλάσης
- Ότι είναι ορισμένο ως private σε μία κλάση είναι προσβάσιμο μόνο από την ίδια κλάση
- Ο τροποποιητής private μας επιτρέπει την απόκρυψη πληροφοριών (information hiding)
  - Ο χρήστης της κλάσης Car, δεν χρειάζεται να ξέρει πως υλοποιείται η μέθοδος `computeRandomSteps` που υπολογίζει τον τυχαίο αριθμό των βημάτων
  - Αν αποφασίσουμε να αλλάξουμε κάτι στη μέθοδο αυτό θα γίνει ως μέρος του επανασχεδιασμού της κλάσης Car. Κανείς άλλος δεν θα πρέπει να επηρεαστεί από την αλλαγή στον κώδικα.
- Τα πεδία μιας κλάσης τα ορίζουμε πάντα private

Βιβλιοθήκες / Πακέτα (packages)

Βιβλιοθήκες / Πακέτα (packages)

- (Συνήθως) Ομαδοποιούμε τις κλάσεις σύμφωνα με τα αντικείμενα που αναπαριστούν και τις λειτουργίες τους.
- Ένα “πακέτο” (βιβλιοθήκη) JAVA απαρτίζεται από μια ομάδα κλάσεων, οι οποίες ανήκουν στον ίδιο χώρο ονομάτων (namespace).
- Για τη δημιουργία μιας βιβλιοθήκης Java, πρέπει να χρησιμοποιήσουμε την λέξη-κλειδί package μαζί με το όνομα της βιβλιοθήκης, στην αρχή κάθε αρχείου, το οποίο θέλουμε να εντάξουμε στην βιβλιοθήκη
- Έτσι, δηλώνοντας : package mypackage στην αρχή κάποιων αρχείων Java, ορίζουμε ότι τα αρχεία αυτά ανήκουν στην ίδια βιβλιοθήκη, η οποία έχει το όνομα mypackage.
- Η δημιουργία μιας βιβλιοθήκης Java («πακέτου») δεν προϋποθέτει ούτε συνεπάγεται την τοποθέτηση αρχείων-κλάσεων σε κάποιο ενιαίο αρχείο.

Βιβλιοθήκες / Πακέτα (packages)

Παράδειγμα: package test.animals;

```
package test.animals;

class Tiger {
  private String name;

  public Tiger(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }
}
```

```
package test.animals;

class Lion {
  private String name;

  public Lion(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }

  public String isKing(){
    return "I am the king";
  }
}
```

Βιβλιοθήκες / Πακέτα (packages)

- Η Java μας δίνει τη δυνατότητα να επαναχρησιμοποιήσουμε τις κλάσεις μιας βιβλιοθήκης JAVA, με χρήση της εντολής import στον κώδικα των αρχείων στα οποία θέλουμε να κάνουμε χρήση των κλάσεων της βιβλιοθήκης

- Τρόπος 1 (χρήση import):

```
import test.animals.Lion;
…
Lion li  = new Lion();
```

- Τρόπος 2 (χωρίς τη χρήση import):

```
test.animals.Lion  li  = new test.animals.Lion();
```

Βιβλιοθήκες / Πακέτα (packages)

```
import test.animals.Lion;
import test.animals.Tiger;
// OR import test.animals.*;

public class TestAnimals{

  public static void main(String args[]){
    Lion l1 = new Lion("lion");
    // OR test.animals.Lion l1 =
                    new test.animals.Lion("lion");

    Tiger t1 = new Tiger("tiger");
    // OR test.animals.Tiger t1 =
                    new test.animals.Tiger(“tiger");
  }
}
```

Βιβλιοθήκες / Πακέτα (packages)

- Ο λόγος της εισαγωγής βιβλιοθηκών στην JAVA, μέσα στον πηγαίο κώδικα, οφείλεται στην ανάγκη διαχείρισης της ονοματολογίας (name spaces) για την αποφυγή συγκρούσεων (name clashes)

- Π.χ. τι θα συμβεί αν ορίσουμε κάποια άλλη κλάση Lion και η ίδια κλάση βρίσκεται ήδη στην μηχανή μας ή φορτώνεται αυτόματα από το δίκτυο καθώς τρέχει μία εφαρμογή μας;

- Για την πρόληψη των προβλημάτων αυτών η JAVA μας δίνει (μέσω των βιβλιοθηκών) πλήρη έλεγχο στην ονοματολογία και τη δυνατότητα να δημιουργούμε απολύτως μοναδικά ονόματα.

Οργάνωση Πακέτων

- Τα αρχεία των κλάσεων (class files) που έχουν δηλωθεί ότι ανήκουν στην ίδια βιβλιοθήκη, πρέπει να τοποθετούνται σε κοινό υποκατάλογο (sub-directory), με όνομα το οποίο συνάγεται από το όνομα της βιβλιοθήκης
- Παράδειγμα: package test.animals ➔<root>/test/animals
- Η διαρρύθμιση αυτή μας επιτρέπει:
  - Να επιλύσουμε το πρόβλημα της ονοματολογίας, χρησιμοποιώντας την ιεραρχική δομή των καταλόγων τού συστήματος αρχείων για να δώσουμε μοναδικά ονόματα σε βιβλιοθήκες - το όνομα της βιβλιοθήκης αντιστοιχεί στην θέση της στο σύστημα αρχείων
  - Να εντοπίζουμε με ευκολία τις κλάσεις μιας βιβλιοθήκης.

Σύγκρουση Ονομάτων

- Ας σε κάποιο πρόγραμμα Java εισάγονται δύο βιβλιοθήκες, οι οποίες περιλαμβάνουν κλάσεις με τα ίδια ονόματα. Π.χ.:
  - import mypackage.util.*;
  - import java.util.*;
  - όπου και οι δύο βιβλιοθήκες ορίζουν την κλάση Vector.

- Δεν υπάρχει πρόβλημα αν δεν χρησιμοποιηθεί η Vector. Διαφορετικά ο μεταφραστής βγάζει μήνυμα λάθους, διότι δεν έχει τρόπο να γνωρίζει για ποιά κλάση πρόκειται

- Αν ωστόσο η κλάση Vector ορίζεται μέσα στο αρχείο μας ή στον κατάλογο όπου αποθηκεύεται το αρχείο μας, τότε ο μεταφραστής υποθέτει ότι οι δύο κλάσεις βρίσκονται στην default βιβλιοθήκη και άρα δεν υπάρχει πρόβλημα σύγκρουσης (θα χρησιμοποιηθεί η τοπική Vector).

Τροποποιητές Πρόσβασης (Access Modifiers)

Τροποποιητές Πρόσβασης (Access Modifiers)

- Μία κλάση ελέγχει την πρόσβαση στα διάφορα μέλη της (δεδομένα, μεθόδους) με τους τροποποιητές πρόσβασης
- Υπάρχουν **4** τροποποιητές πρόσβασης:
  - **public**: Η πρόσβαση στην κλάση, δεδομένα, μεθόδους είναι ανοικτή σε όλους!
  - **private**: Η πρόσβαση στα δεδομένα και τις μεθόδους είναι περιορισμένη μόνο στην κλάση
  - **default (friendly)**: Η πρόσβαση στην κλάση, δεδομένα και μεθόδους είναι περιορισμένη στις κλάσεις που βρίσκονται στο ίδιο πακέτο με την κλάση
  - **protected**: Όπως το default αλλά και σε κλάσεις που κληρονομούν από την ίδια κλάση (εντός πακέτου ή όχι)

Τροποποιητές Πρόσβασης (Access Modifiers)

- Σε περίπτωση που δεν χρησιμοποιείται προσδιοριστής πρόσβασης, έχουμε τον **προκαθορισμένο (default)** προσδιοριστή πρόσβαση, ο οποίος αναφέρεται σαν «φιλικός» (friendly):
  - Ο φιλικός προσδιοριστής υποδηλώνει ότι όλες οι κλάσεις σε ένα πακέτο (βιβλιοθήκη) έχουν πρόσβαση στα «φιλικά» μέλη, τα οποία εμφανίζονται ως «**private**» σε όλες τις κλάσεις έξω από την βιβλιοθήκη
  - Αφού μια κλάση μπορεί να ανήκει σε μια μόνο βιβλιοθήκη, όλες οι κλάσεις μέσα σε αυτή θεωρούνται αυτομάτως φιλικές η μια προς την άλλη

Τροποποιητές Πρόσβασης (Access Modifiers)

- Για να δοθεί πρόσβαση προς τα μέλη μιας κλάσης, από άλλες κλάσεις, υπάρχουν οι εξής εναλλακτικές δυνατότητες:
  - Να καταστεί το μέλος της κλάσης που μας ενδιαφέρει δημόσιο (**public**). Όλοι μπορούν να έχουν πρόσβαση – **Δεν συνίσταται**
  - Να γίνει το μέλος της κλάσης που μας ενδιαφέρει φιλικό, οριζόμενο χωρίς κανέναν προσδιοριστή – και οι υπόλοιπες κλάσεις να τοποθετηθούν στην ίδια βιβλιοθήκη
  - Στην περίπτωση κληρονομικότητας, με χρήση του προσδιοριστή protected
  - Με δήλωση μεθόδων πρόσβασης/τροποποίησης (accessor/mutator), οι οποίες διαβάζουν και γράφουν τιμές της κλάσης που μας αφορά

Τροποποιητές Πρόσβασης: Σύγκριση

Τροποποιητής Πρόσβασης	Ίδια Κλάση	Ίδιο Πακέτο	Υποκλάση	Άλλα Πακέτα
public	√	√	√	√
protected	√	√	√	
(default) friendly	√	√		
private	√			

Τροποποιητές Πρόσβασης – Καλές Πρακτικές

- Όταν κατασκευάζετε μια κλάση είναι σκόπιμο να δηλώσετε τα πεδία δεδομένων της σαν ιδιωτικά, για να τα προστατεύσετε από ανεπιθύμητες αλλαγές από τρίτους
- Αυτό ισχύει ακόμα κι αν η κλάση σας είναι προσβάσιμη από τη βιβλιοθήκη της μόνο (όχι δημόσια)
- Επίσης, είναι συνήθως λογικό να δώσετε στις μεθόδους της κλάσης την ίδια προσβασιμότητα με την κλάση, δηλ. δημόσια ή φιλική
- Μπορείτε να κάνετε (κάποιους) κατασκευαστές της κλάσης private

Τροποποιητές Πρόσβασης για Κλάσεις

- Οι προσδιοριστές πρόσβασης μπορούν να χρησιμοποιηθούν και για τον καθορισμό των κλάσεων μιας βιβλιοθήκης που είναι προσβάσιμες είτε μέσα από την ίδια βιβλιοθήκη είτε από άλλες βιβλιοθήκες
- Μια κλάση μπορεί να προσδιορισθεί μόνο σαν φιλική ή σαν δημόσια.
  - public: ΟΛΟΙ έχουν πρόσβαση στην κλάση
  - <no modifier>: Μόνο κλάσεις στο ίδιο πακέτο έχουν πρόσβαση στην κλάση
- Κανονικές (μη εσωτερικές) κλάσεις δεν μπορούν να συνδυαστούν με προσδιοριστές πρόσβασης private ή protected

Αφαιρετικότητα (abstraction)

Αφαιρετικότητα (abstraction)

- Η αφαιρετικότητα είναι επιλεκτική άγνοια!
- Επιλογή του τι είναι σημαντικό και τι όχι
  - Έμφαση και Εξάρτηση στα σημαντικά
  - Αγνόηση και Απεξάρτηση από τα ΜΗ σημαντικά
- Χρήση της ενθυλάκωσης (encapsulation) για να επιτύχουμε την αφαιρετικότητα

Παράδειγμα Αφαιρετικότητας - Κινητά τηλέφωνα

- Παρατηρήσεις
  - Πολλά HTC τηλέφωνα
  - 1 iPhone
  - Μοντέλα για Ψυχαγωγία
  - Μοντέλα για Δουλειά
  - Μοντέλα για Κοινωνικότητα
  - Touch ή με keypad
  - Διαφορετικά χρώματα, βάρος…
  - … και πολλά άλλα

- Όλα είναι κινητά τηλέφωνα



Παράδειγμα Αφαιρετικότητας - Κινητά τηλέφωνα

Όλα είναι κινητά τηλέφωνα



Παράδειγμα Αφαιρετικότητας - Κινητά τηλέφωνα

Χρειαζόμαστε όμως καλύτερη κατηγοριοποίηση…

Ας τα κατηγοριοποιήσουμε ανάλογα με τον κατασκευαστή τους



Παράδειγμα Αφαιρετικότητας - Κινητά τηλέφωνα

Θα μπορούσαμε όμως να τα κατηγοριοποιήσουμε και σύμφωνα με τη χρήση τους π.χ., εργασία, ψυχαγωγία, κοινωνικότητα.



Παράδειγμα Αφαιρετικότητας - Κινητά τηλέφωνα

Θα μπορούσαμε όμως να τα κατηγοριοποιήσουμε και σύμφωνα με το αν είναι touch ή type

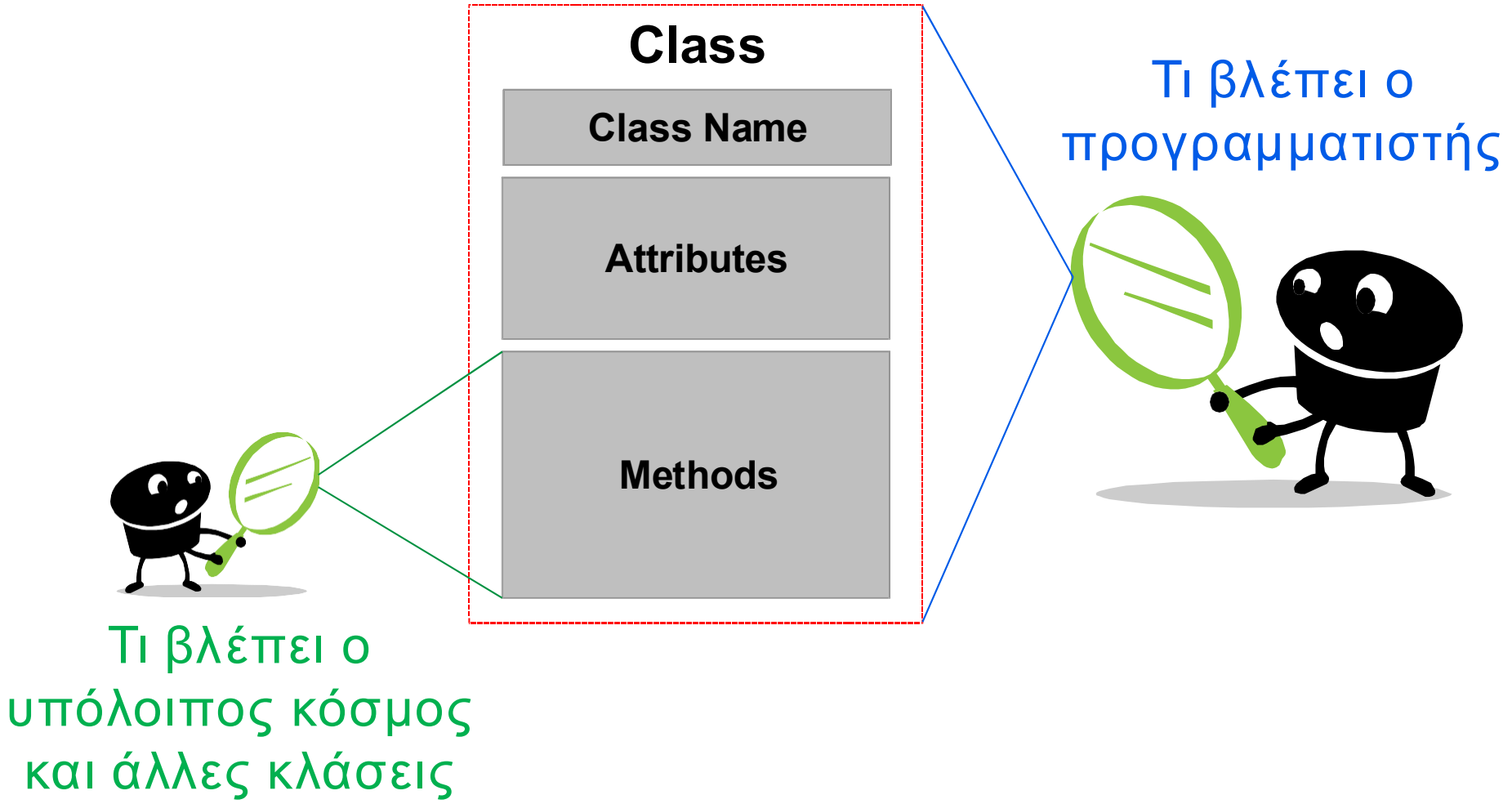


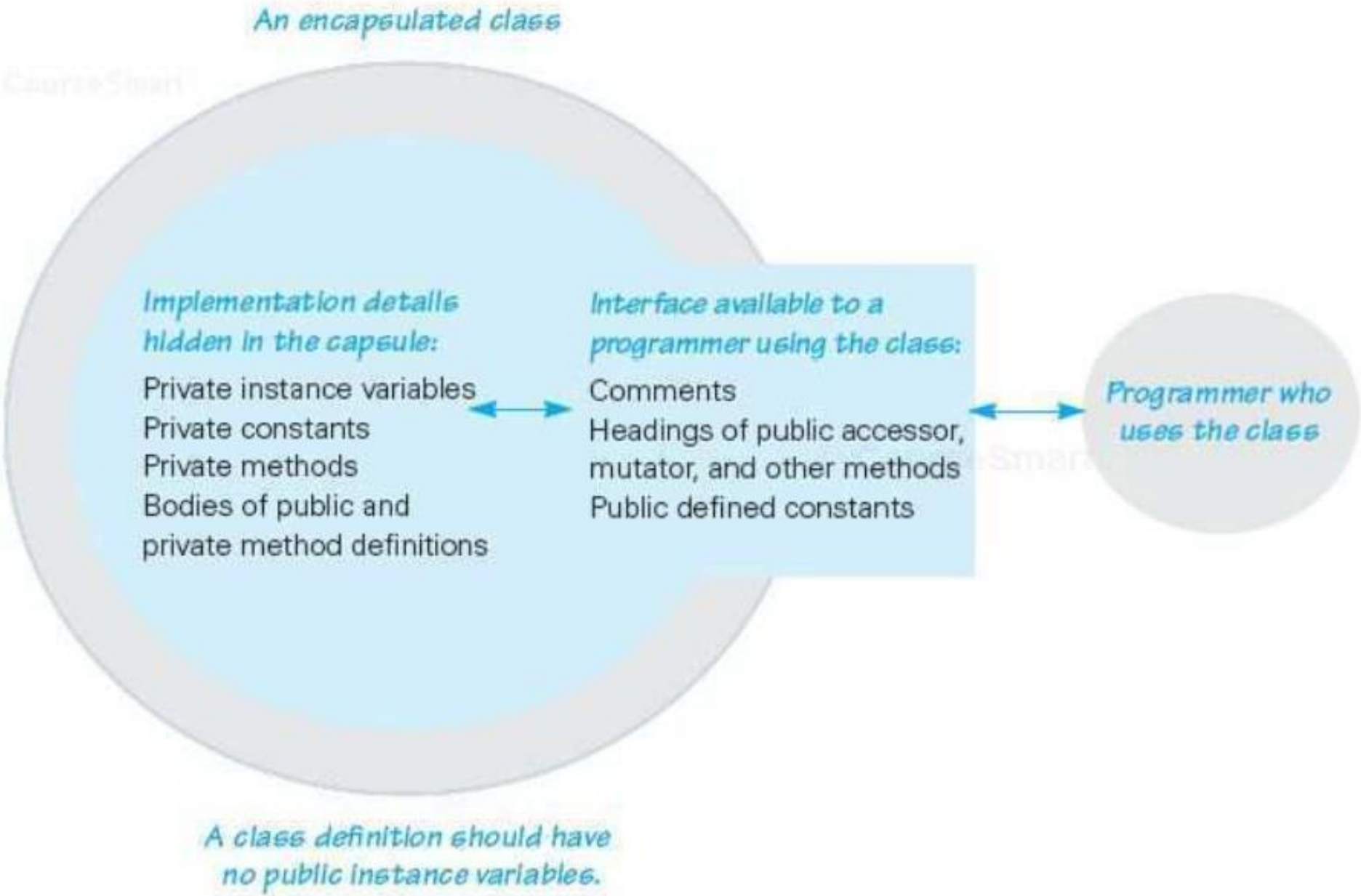
Ενθυλάκωση (encapsulation)

Ενθυλάκωση (encapsulation)

- Η ομαδοποίηση λογισμικού και δεδομένων σε μία οντότητα (κλάση και αντικείμενα της κλάσης) ώστε να είναι εύχρηστη μέσω ενός καλά ορισμένου interface, ενώ οι λεπτομέρειες υλοποίησης είναι κρυμμένες από τον χρήστη
- API (Application Programming Interface)
  - Μια περιγραφή για το πώς χρησιμοποιείται η κλάση μέσω των public μεθόδων της
    - Java docs είναι ένα παράδειγμα
- Το API είναι αρκετό για να χρησιμοποιήσετε μια κλάση, δεν χρειάζεται να ξέρετε την υλοποίηση των μεθόδων

Ενθυλάκωση (encapsulation)




An encapsulated class
Implementation details hidden in the capsule:
Private instance variables
Private constants
Private methods
Bodies of public and private method definitions
Interface available to a programmer using the class:
Comments
Headings of public accessor, mutator, and other methods
Public defined constants
Programmer who uses the class
A class definition should have no public instance variables.

Accessor and Mutator methods

- Πολλές φορές χρειαζόμαστε να διαβάσουμε ή να αλλάξουμε ένα πεδίο ενός αντικειμένου
  - Π.χ., να διαβάσουμε τη θέση του οχήματος, ή να τοποθετήσουμε το όχημα σε συγκεκριμένη θέση
  - Πως θα το κάνουμε αφού τα πεδία είναι private?

- Ορίζουμε ειδικές μεθόδους
  - Μέθοδος προσπέλασης (**accessor method**) για διάβασμα
  - Μέθοδος μεταλλαγής (**mutator method**) για γράψιμο

- **Σύμβαση**: Στη Java η ονοματολογία των μεθόδων αυτών γίνεται με συγκεκριμένο τρόπο:
  - **get<ονομα μεταβλητης>** για την πρόσβαση
    - getPosition
  - **set<ονομα μεταβλητης>** για την μετάλλαξη
    - setPosition

```
class Car
{
  private int position = 0;

  public void setPosition(int position){
    this.position = position;
  }

  public int getPosition(){
    return position;
  }

  public void move(){
    position ++ ;
  }
}

class MovingCar7
{
  public static void main(String args[]){
    Car myCar = new Car();
    myCar.setPosition(10);
    myCar.move();
    System.out.println(myCar.getPosition());
  }
}
```

Υπάρχουν περιπτώσεις που μπορεί να θέλουμε η συνάρτηση set να επιστρέφει boolean (true αν η ανάθεση έγινε επιτυχώς, false αλλιώς)

```
class Car
{
        private int position = 0;

        public boolean setPosition(int position){
                if (position < 0){
                        return false;
                }
                this.position = position;
                return true;
        }

        public int getPosition(){
                return position;
        }

        public void move(){
                position ++ ;
        }
}

class MovingCar9
{
        public static void main(String args[]){
                Car myCar = new Car();
                boolean check = myCar.setPosition(-1);
                if (!check){
                        System.out.println("position not set");
                }
        }
}
```

Keyword “this”

- Οι τοπικές μεταβλητές έχουν προτεραιότητα σε σχέση με τα πεδία του αντικειμένου
  - Αν έχουμε μια τοπική μεταβλητή με το ίδιο όνομα με ένα πεδίο μέσα σε μία μέθοδο (πεδίο αντικειμένου), όταν χρησιμοποιούμε το όνομα αναφερόμαστε στην τοπική μεταβλητή και όχι στο πεδίο
  - Αν θέλουμε να αναφερθούμε στο πεδίο μπορούμε να χρησιμοποιήσουμε την δεσμευμένη λέξη this. Το this αναφέρεται πάντα στο αντικείμενο.

```
class Car
{
  private int position = 0;

  public void setPosition(int position){
    this.position = position;
  }

  public int getPosition(){
    return position;
  }

  public void move(){
    position ++ ;
  }
}

class MovingCar7
{
  public static void main(String args[]){
    Car myCar = new Car();
    myCar.setPosition(10);
    myCar.move();
    System.out.println(myCar.getPosition());
  }
}
```

Το this.position αναφέρεται στο πεδίο του αντικειμένου.
Το position αναφέρεται στην παράμετρο της συνάρτησης

Το κρυφό πεδίο this προσδιορίζει το αντικείμενο που κάλεσε την μέθοδο

Έτσι μπορούμε να χρησιμοποιήσουμε το ίδιο όνομα μεταβλητής χωρίς να δημιουργείται σύγχυση

```
class LocalVariableTest
{
        private int var = 10;

        public void method1(){
                int var = 5;
                var ++;
        }

        public void method2(int var){
                var ++;
        }

        public void method3(){
                int var = 1;
                this.var = var;
        }

        public void printVar(){
                System.out.println("var = "+var);
        }

        public static void main(String[] args){
                LocalVariableTest x = new LocalVariableTest();
                x.method1(); x.printVar();
                x.method2(3); x.printVar();
                x.method3(); x.printVar();
        }
}
```

Ορισμός του πεδίου var

Ορισμός τοπικής μεταβλητής var.
Η χρήση της var μέσα στην μέθοδο αναφέρεται στην τοπική μεταβλητή

Ορισμός παραμέτρου var.
Η χρήση της var μέσα στην μέθοδο αναφέρεται στην τοπική μεταβλητή

Ορισμός τοπικής μεταβλητής var.
Η χρήση της var μεσα στην μέθοδο αναφέρεται στην τοπική μεταβλητή.
Το this.var αναφέρεται στο πεδίο της κλάσης

Παρένθεση: Μπορούμε να ορίσουμε main μέσα σε μία κλάση για να την τεστάρουμε

Τι θα τυπώσει?
var = 10
var = 10
var = 1

Constructors (Κατασκευαστές)

- Όταν δημιουργούμε ένα αντικείμενο συχνά θέλουμε να μπορούμε να το αρχικοποιήσουμε με κάποιες τιμές
  - Ένα **Person** να αρχικοποιείται με ένα **όνομα**
  - Ένα **Car** να αρχικοποιείται με μία **θέση**
- Μπορούμε να το κάνουμε με μία συνάρτηση set αυτό, αλλά
  - Μπορεί να έχουμε πολλές μεταβλητές να αρχικοποιήσουμε
  - Θέλουμε η αρχικοποίηση να είναι μέρος της δημιουργίας του αντικειμένου
- Την αρχικοποίηση μπορούμε να την κάνουμε με ένα **Constructor** (**Κατασκευαστή**)

Constructors (Κατασκευαστές)

- Ο **Constructor** είναι μια «μέθοδος» η οποία καλείται όταν **δημιουργούμε** το αντικείμενο χρησιμοποιώντας την new
- Αν δεν έχουμε ορίσει Constructor καλείται ένας default Constructor χωρίς ορίσματα που δεν κάνει τίποτα
- Αν ορίσουμε constructor, τότε καλείται ο constructor που ορίσαμε

Παράδειγμα

```
class Person
{
    private String name;

    public Person(String name){
       this.name = name;
    }

    public void speak(String s){
       System.out.println(name+": "+s);
    }
}

public class HelloWorld2
{
    public static void main(String[] args){
       Person alice = new Person("Alice");
       alice.speak("Hello World");
    }
}
```

Constructor: μια μέθοδος με το ίδιο όνομα όπως και η κλάση και χωρίς τύπο (ούτε void)

Αρχικοποιεί την μεταβλητή name

Constructor: καλείται όταν δημιουργείται το αντικείμενο με την new και μόνο τότε

Μια Συνομιλία

```
class Person
{
    private String name;

    public Person(String name){
        this.name = name;
    }

    public void speak(String s){
        System.out.println(name+": "+s);
    }
}

public class Conversation
{
    public static void main(String[] args){
        Person alice = new Person("Alice");
        Person bob = new Person("Bob");
        alice.speak("Hi Bob");
        bob.speak("Hi Alice");
    }
}
```

```
Alice: Hi Bob
Bob: Hi Alice
```

Παράδειγμα

```
class Car
{
        private int position;

        public Car(int position){
                this.position = position;
        }

        public void move(int delta){
                position += delta ;
        }

        public void printPosition(){
                System.out.println("Car is at position "+position);
        }

}

class MovingCar9
{
        public static void main(String args[]){
                Car myCar1 = new Car(1);
                Car myCar2 = new Car(-1);
                myCar1.move(-1); myCar1.printPosition();
                myCar2.move(1); myCar2.printPosition();
        }
}
```

```
Car is at position 0
Car is at position 0
```

Παράδειγμα

```
class Car
{
  private int position = 0;
  private int ACCELERATOR = 2;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += ACCELERATOR * delta ;
  }

  public void printPosition(){
    System.out.println("Car is at position "+position);
  }
}

class MovingCar10
{
  public static void main(String args[]){
          Car myCar1 = new Car(1);
          Car myCar2 = new Car(-1);
          myCar1.move(-1); myCar1.printPosition();
          myCar2.move(1); myCar2.printPosition();
  }
}
```

Η εκτέλεση αυτών των αρχικοποιήσεων γίνεται πριν εκτελεστούν οι εντολές στον constructor

Η τελική τιμή του position θα είναι αυτή που δίνεται σαν όρισμα

```
Car is at position -1
Car is at position 1
```

Υπερφόρτωση

- Είδαμε μια περίπτωση που είχαμε μια συνάρτηση *move* η οποία μετακινεί το όχημα κατά μία θέση, και μια συνάρτηση *moveManySteps* η οποία το μετακινεί όσες θέσεις ορίζει το όρισμα
  - Το να θυμόμαστε δυο ονόματα είναι μπερδεμένο, θα ήταν καλύτερο να είχαμε μόνο ένα. Και στις δύο περιπτώσεις η λειτουργία που θέλουμε να κάνουμε είναι move
- Η Java μας δίνει αυτή τη δυνατότητα μέσω της διαδικασίας της υπερφόρτωσης (overloading)
  - Ορισμός πολλών μεθόδων με το **ίδιο όνομα** αλλά **διαφορετικά ορίσματα**, μέσα στην ίδια κλάση

```
class Car
{
        private int position;

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar11
{
        public static void main(String args[]){
                Car myCar = new Car(1);
                myCar.move();
                myCar.move(-1);
        }
}
```

Υπερφόρτωση Constructor

- Είναι αρκετά συνηθισμένο να υπερφορτώνουμε τους constructors των κλάσεων

```
class Car
{
        private int position;

        public Car(){
                this.position = 0;
        }

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar12
{
        public static void main(String args[]){
                Car myCar1 = new Car(1); myCar1.move();
                Car myCar2= new Car(); myCar2.move(-1);
        }
}
```

```
class Car
{
        private int position = 0;

        public Car(){}

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar12
{
        public static void main(String args[]){
                Car myCar1 = new Car(1); myCar1.move();
                Car myCar2= new Car(); myCar2.move(-1);
        }
}
```

Κενός κώδικας, χρειάζεται για να οριστεί ο “default” constructor

Γενικά είναι καλό να ορίζετε και ένα constructor χωρίς ορίσματα

Υπερφόρτωση – Προσοχή Ι

- Όταν ορίζουμε ένα constructor, ο default constructor παύει να υπάρχει. Πρέπει να τον ορίσουμε μόνοι μας.

```
class Car
{
        private int position = 0;

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar12
{
        public static void main(String args[]){
                Car myCar1 = new Car(1);
                myCar1.move();
                Car myCar2= new Car();
                myCar2.move(-1);
        }
}
```

Θα χτυπήσει λάθος ότι δεν υπάρχει constructor χωρίς ορίσματα

Υπερφόρτωση – Προσοχή II

- Η υπερφόρτωση γίνεται μόνο ως προς τα ορίσματα, ΟΧΙ ως προς την επιστρεφόμενη τιμή
- Η **υπογραφή** μίας μεθόδου είναι το όνομα της και η λίστα με τους τύπους των ορισμάτων της μεθόδου
  - Η Java μπορεί να ξεχωρίσει μεθόδους με διαφορετική υπογραφή
  - Π.χ., move(), move(int) έχουν διαφορετική **υπογραφή**
- Όταν δημιουργούμε μια μέθοδο θα πρέπει να δημιουργούμε μία διαφορετική υπογραφή

```
class SomeClass
{
  public int aMethod(int x, double y){
    System.out.println("int double");
    return 1;
  }

  public double aMethod(int x, double y){
    System.out.println("int double");
    return 1;
  }

  public int aMethod(double x, int y){
    System.out.println("double int");
    return 1;
  }

  public double aMethod(double x, int y){
    System.out.println("double int");
    return 1;
  }

}
```

A

B

C

D

Ποιοι συνδυασμοί είναι αποδεκτοί?



Υπερφόρτωση – Προσοχή III

- Λόγω της συμβατότητας μεταξύ τύπων μια κλήση μπορεί να ταιριάζει με διάφορες μεθόδους
- Καλείται αυτή που ταιριάζει ακριβώς, ή αυτή που είναι πιο κοντά
- Αν υπάρχει **ασάφεια** θα χτυπήσει ο compiler

```
class SomeClass
{

  public int aMethod(int x, int y){
    System.out.println("int int");
    return 1;
  }

  public float aMethod(float x, float y){
    System.out.println("float float");
    return 1;
  }

  public double aMethod(double x, double y){
    System.out.println("double double");
    return 1;
  }
}

class OverloadingExample
{
  public static void main(String args[])
  {
    SomeClass anObject = new SomeClass();
    anObject.aMethod(1,1);
  }
}
```

Τι θα τυπώσει η κλήση της μεθόδου?

Τυπώνει “int int” γιατί ταιριάζει ακριβώς με τις παραμέτρους που δώσαμε

```
class SomeClass
{
  /*
  public int aMethod(int x, int y){
    System.out.println("int int");
    return 1;
  }
  */

  public float aMethod(float x, float y){
    System.out.println("float float");
    return 1;
  }

  public double aMethod(double x, double y){
    System.out.println("double double");
    return 1;
  }
}

class OverloadingExample
{
  public static void main(String args[])
  {
    SomeClass anObject = new SomeClass();
    anObject.aMethod(1,1);
  }
}
```

Τι θα τυπώσει η κλήση της μεθόδου?

Τυπώνει “float float” γιατί είναι πιο κοντά ακριβώς με τις παραμέτρους που δώσαμε

Ασάφεια

```
class SomeClass
{
  public double aMethod(int x, double y){
    System.out.println("int double");
    return 1;
  }

  public int aMethod(double x, int y){
    System.out.println("double int");
    return 1;
  }
}

class OverloadingExample
{
  public static void main(String args[])
  {
    SomeClass anObject = new SomeClass();
    anObject.aMethod(1.0,1);
    anObject.aMethod(1,1);
  }
}
```

Τι θα τυπώσει η κλήση της μεθόδου σε κάθε περίπτωση?

Τυπώνει “double int”

Ο compiler μας πετάει λάθος γιατί η κλήση είναι ασαφής (ambiguous)

Αντικείμενα ως ορίσματα

- Μπορούμε να περνάμε αντικείμενα ως ορίσματα σε μία μέθοδο όπως οποιαδήποτε άλλη μεταβλητή
- Οποιαδήποτε κλάση μπορεί να χρησιμοποιηθεί ως παράμετρος
- Όταν τα ορίσματα ανήκουν στην κλάση στην οποία ορίζεται η μέθοδος τότε η μέθοδος μπορεί να δει (και) τα **ιδιωτικά** (private) πεδία των αντικειμένων
- Αν τα ορίσματα είναι διαφορετικού τύπου τότε η μέθοδος μπορεί μόνο να καλέσει τις public μεθόδους

Παράδειγμα

- Ορίστε μια μέθοδο που να μας επιστρέφει την απόσταση μεταξύ δύο οχημάτων

```
class Car
{
  private int position = 0;

  public Car(int position){
    this.position = position;
  }

  public int getPosition() {return position;}

  public void move(int delta){
    position += delta ;
  }
}

class MovingCarDistance1
{
  public static void main(String args[]){
    Car myCar1 = new Car(1);
    Car myCar2 = new Car(0);
    myCar2.move(2);
    System.out.println("Distance of Car 1 from Car 2: " + computeDistance(myCar1,myCar2));
    System.out.println("Distance of Car 2 from Car 1: " + computeDistance(myCar2,myCar1));
  }

  private static int computeDistance(Car car1, Car car2){
    return car1.getPosition() - car2.getPosition();
  }
}
```

Μια μέθοδος ή ένα πεδίο που χρησιμοποιείται σε μία static μέθοδο πρέπει να είναι επίσης static

Η μέθοδος computeDistance παίρνει σαν όρισμα δύο αντικείμενα τύπου Car

```
class Car
{
  private int position = 0;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += delta ;
  }

  public int distanceFrom(Car other){
    return this.position - other.position;
  }
}

class MovingCarDistance2
{
  public static void main(String args[]){
    Car myCar1 = new Car(1);
    Car myCar2 = new Car(0); myCar2.move(2);
    System.out.println("Distance of Car 1 from Car 2: " + myCar1.distanceFrom(myCar2));
    System.out.println("Distance of Car 2 from Car 1: " + myCar2.distanceFrom(myCar1));
}
```

Συνήθως προτιμούμε όποια μέθοδος έχει σχέση με την κλάση να την ορίζουμε ως public μέθοδο της κλάσης. Έχουμε επιπλέον ευελιξία γιατί έχουμε πρόσβαση σε όλα τα πεδία της κλάσης

Αν και το πεδίο position είναι private μπορούμε να το προσπελάσουμε γιατί είμαστε μέσα στην κλάση Car. Μία κλάση μπορεί να προσπελάσει τα ιδιωτικά μέλη όλων των αντικειμένων της κλάσης